# Bài 2.8 – Naive Bayes: lọc tin nhắn spam

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 1 buổi (2–3 giờ) · Hình thức mở đầu: *Tìm lỗi trong code*

## 🎯 Sau bài này bạn sẽ

- Hiểu Naive Bayes = **định lý Bayes (bài 1.10) + một giả định "ngây thơ"**.
- Tự tính tay xác suất spam của một tin nhắn.
- Biết vì sao cần **Laplace smoothing**.
- Xây bộ lọc spam trên **5.572 tin nhắn SMS thật**, đạt độ chính xác gần 99%.
- Biến văn bản thành số bằng `CountVectorizer`, và tránh lỗi kinh điển khi dùng nó.

---

## 1. Tìm lỗi trong code 🐞 (10 phút)

Bạn Minh viết bộ lọc spam. Code chạy đến dòng cuối thì báo lỗi. Đọc kỹ và tìm lỗi **trước khi** đọc tiếp.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

vec = CountVectorizer()
X_train_so = vec.fit_transform(X_train)                # biến tin nhắn train
thành số
X_test_so = CountVectorizer().fit_transform(X_test)    # biến tin nhắn test thành
số

mo_hinh = MultinomialNB().fit(X_train_so, y_train)
mo_hinh.predict(X_test_so)                              # ❌ lỗi ở đây
```

Thông báo lỗi (rút gọn):

```
ValueError: X has 3590 features, but MultinomialNB is expecting 7668 features as
input.
```

Gợi ý: con số 3590 và 7668 từ đâu ra? Đáp án ở mục 5.3. Sau khi học xong bài, bạn sẽ thấy lỗi này **rất hiển nhiên**.

---

## 2. Dữ liệu: SMS Spam Collection

Bộ dữ liệu thật gồm 5.572 tin nhắn SMS tiếng Anh, mỗi tin đã được gán nhãn `ham` (tin bình thường) hoặc `spam`.

```
import numpy as np
import pandas as pd

url = "https://raw.githubusercontent.com/justmarkham/pycon-2016-
tutorial/master/data/sms.tsv"
sms = pd.read_csv(url, sep="\t", header=None, names=["nhan", "tin"])   # file ngăn
cách bằng Tab
print(sms.shape)
print(sms["nhan"].value_counts(normalize=True).round(3).to_dict())
print(sms.head(3).to_string())
```

**Kết quả mẫu:**

```
(5572, 2)
{'ham': 0.866, 'spam': 0.134}
   nhan
tin
0   ham                                               Go until jurong point,
crazy.. Available only in bugis n great world la e buffet... Cine there got amore
wat...
1   ham
Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text
FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's
```

13,4% là spam. Dữ liệu **mất cân bằng** → nhớ bài 2.5: đừng chỉ nhìn accuracy.

> Vì sao dùng tiếng Anh? Tiếng Việt cần thêm bước **tách từ** ("học sinh" là 1 từ, không phải 2), sẽ học kỹ ở bài NLP1.1. Ý tưởng Naive Bayes thì giống hệt.

---

# 3. Biến văn bản thành số: Bag of Words

## 3.1. Khái niệm

- **Bag of Words (Túi từ):** Biểu diễn mỗi văn bản bằng **số lần xuất hiện của từng từ**, bỏ qua thứ tự từ. (Sẽ học kỹ ở bài NLP1.3.)
- **Ví dụ đời thực:** Đổ hết chữ trong một lá thư vào cái túi, xóc đều, rồi đếm: "free" 2 lần, "prize" 1 lần... Mất thứ tự, nhưng vẫn đoán được đó là thư quảng cáo.

```
 "free prize now"        ⟶  call free lunch me meet money now prize tomorrow win
 "free free money"       ⟶   0    1    0    0   0    0    1    1      0      0
                             0    2    0    0   0    1    0    0      0      0
```

## 3.2. `CountVectorizer`

`CountVectorizer` là một **transformer** (bài 2.1):

- `fit`: học **bộ từ vựng** (danh sách mọi từ xuất hiện trong dữ liệu train).
- `transform`: đếm từ của mỗi văn bản **theo đúng bộ từ vựng đó**.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(sms["tin"], sms["nhan"], test_size=0.2,
                                          random_state=42, stratify=sms["nhan"])
vec = CountVectorizer()                 # mặc định: đổi về chữ thường, bỏ dấu câu,
bỏ từ 1 ký tự
Xtr = vec.fit_transform(X_tr)           # HỌC từ vựng trên train + đếm
Xte = vec.transform(X_te)               # CHỈ đếm, dùng từ vựng của train
print(Xtr.shape, Xtr.nnz, round(Xtr.nnz / (Xtr.shape[0] * Xtr.shape[1]) * 100, 3),
"%")
```

**Kết quả mẫu:**

```
(4457, 7668) 58930 0.172 %
```

- 4.457 tin nhắn × **7.668 từ** khác nhau.
- Chỉ 58.930 ô khác 0, tức **0,17%**: mỗi tin nhắn chỉ dùng vài chục từ trong 7.668 từ. Bảng gần như toàn số 0, gọi là **ma trận thưa (sparse matrix)**. Scikit-learn chỉ lưu các ô khác 0 để tiết kiệm bộ nhớ.

---

# 4. Naive Bayes

## 4.1. Ý tưởng

Từ bài 1.10, định lý Bayes:

$$P(\text{spam} \mid \text{tin nhắn}) = \frac{P(\text{tin nhắn} \mid \text{spam}) \times P(\text{spam})}{P(\text{tin nhắn})}$$

- P(spam): tỉ lệ spam trong dữ liệu (prior).
- P(tin nhắn | spam): khả năng spam "viết ra" đúng tin nhắn này. **Khó tính**: gần như không có hai tin nhắn giống hệt nhau.

## 4.2. Giả định "ngây thơ"

**Naive (Ngây thơ):** giả định các từ xuất hiện **độc lập** với nhau khi đã biết lớp. Khi đó:

$$P(\text{tin nhắn} \mid \text{spam}) \approx P(\text{từ}_1 \mid \text{spam}) \times P(\text{từ}_2 \mid \text{spam}) \times \ldots \times P(\text{từ}_n \mid \text{spam})$$

Mỗi P(từ | spam) thì dễ tính: đếm xem từ đó chiếm bao nhiêu phần trong tất cả các từ của tin spam.

**Vì sao "ngây thơ"?** Thật ra các từ **không** độc lập: thấy "free" thì khả năng gặp "prize" cao hơn. Giả định này sai, nhưng mô hình vẫn hoạt động **rất tốt** cho văn bản. Vì ta chỉ cần biết lớp nào có điểm **cao hơn**, không cần xác suất chính xác tuyệt đối.

```
So sánh hai "điểm":
  Điểm spam = P(spam) × P(từ₁|spam) × P(từ₂|spam) × ...
  Điểm ham  = P(ham)  × P(từ₁|ham)  × P(từ₂|ham)  × ...
  Lớp nào điểm cao hơn → thắng. Chia mỗi điểm cho tổng hai điểm → ra xác suất.
```

## 4.3. Laplace smoothing (Làm trơn Laplace)

**Vấn đề:** Nếu một từ **chưa từng xuất hiện** trong tin spam nào của dữ liệu train (ví dụ "tomorrow"), thì P("tomorrow" | spam) = 0. Nhân với 0 → điểm spam = 0, **bất kể** tin nhắn có bao nhiêu từ "free", "prize". Một từ duy nhất phủ quyết tất cả!

**Cách chữa:** cộng thêm α (thường là 1) vào mọi số đếm, như thể mỗi từ đã xuất hiện thêm α lần.

$$P(\text{từ} \mid \text{spam}) = \frac{\text{số lần từ xuất hiện trong spam} + \alpha}{\text{tổng số từ trong spam} + \alpha \times \text{kích thước từ vựng}}$$

## 4.4. Tính tay

Dữ liệu đồ chơi 6 tin nhắn:

| Tin nhắn | Nhãn |
|---|---|
| free prize now | spam |
| free free money | spam |
| win prize | spam |
| meet now | ham |
| lunch meet tomorrow | ham |
| call me tomorrow | ham |

Tin mới: **"free prize tomorrow"**. Spam hay ham?

**Bước 1: đếm.** Từ vựng có 10 từ. Tổng số từ trong spam: 3 + 3 + 2 = 8. Trong ham: 2 + 3 + 3 = 8. P(spam) = P(ham) = 3/6 = 0,5.

| Từ | Đếm trong spam | P(từ \| spam) với α = 1 | Đếm trong ham | P(từ \| ham) với α = 1 |
|---|---|---|---|---|
| free | 3 | (3+1)/(8+10) = 4/18 ≈ 0,2222 | 0 | 1/18 ≈ 0,0556 |
| prize | 2 | 3/18 ≈ 0,1667 | 0 | 1/18 ≈ 0,0556 |
| tomorrow | **0** | 1/18 ≈ 0,0556 | 2 | 3/18 ≈ 0,1667 |

**Bước 2: tính điểm.**

- Điểm spam = 0,5 × 0,2222 × 0,1667 × 0,0556 ≈ **0,001029**
- Điểm ham = 0,5 × 0,0556 × 0,0556 × 0,1667 ≈ **0,000257**

**Bước 3: chuẩn hóa.** P(spam) = 0,001029 / (0,001029 + 0,000257) ≈ **0,80**.

Không có smoothing, "tomorrow" (chưa từng xuất hiện trong spam) sẽ làm điểm spam = 0 → kết luận "100% ham". Sai rõ ràng!

```
from sklearn.naive_bayes import MultinomialNB

toy = pd.DataFrame({"tin": ["free prize now", "free free money", "win prize",
                            "meet now", "lunch meet tomorrow", "call me
tomorrow"],
                    "nhan": ["spam", "spam", "spam", "ham", "ham", "ham"]})
cv = CountVectorizer()
Xt = cv.fit_transform(toy["tin"])
print(cv.get_feature_names_out())
print(Xt.toarray())                                  # toarray: đổi ma trận thưa
thành bảng đầy đủ để xem
m = MultinomialNB(alpha=1).fit(Xt, toy["nhan"])      # alpha = hệ số Laplace
smoothing
print(m.predict_proba(cv.transform(["free prize tomorrow"])).round(4), m.classes_)
```

**Kết quả mẫu:**

```
['call' 'free' 'lunch' 'me' 'meet' 'money' 'now' 'prize' 'tomorrow' 'win']
[[0 1 0 0 0 0 1 1 0 0]
 [0 2 0 0 0 1 0 0 0 0]
 [0 0 0 0 0 0 0 1 0 1]
 [0 0 0 0 1 0 1 0 0 0]
 [0 0 1 0 1 0 0 0 1 0]
 [1 0 0 1 0 0 0 0 1 0]]
[[0.2 0.8]] ['ham' 'spam']
```

Khớp với tính tay: **80% spam**.

> 💡 Với văn bản dài, nhân hàng trăm số nhỏ sẽ ra số nhỏ đến mức máy tính làm tròn thành 0. Thực tế, thư viện cộng các **logarit** thay vì nhân xác suất: log(a × b) = log(a) + log(b).

---

# 5. Bộ lọc spam thật

## 5.1. Huấn luyện và đánh giá

```
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

nb = MultinomialNB().fit(Xtr, y_tr)        # alpha mặc định = 1
d = nb.predict(Xte)
print(round(accuracy_score(y_te, d), 4))
print(confusion_matrix(y_te, d, labels=["ham", "spam"]))
print(classification_report(y_te, d, digits=3))
```

**Kết quả mẫu:**

```
0.9874
[[964   2]
 [ 12 137]]
              precision    recall  f1-score   support

         ham      0.988     0.998     0.993       966
        spam      0.986     0.919     0.951       149

    accuracy                          0.987      1115
   macro avg      0.987     0.959     0.972      1115
weighted avg      0.987     0.987     0.987      1115
```

**Đọc theo bài 2.5:**

- Mốc "đoán toàn ham" đạt 86,6%. Mô hình đạt **98,7%**.
- Precision spam **0,986**: chỉ **2** tin bình thường bị đưa nhầm vào spam. Rất quan trọng: không ai muốn tin nhắn của mẹ bị chặn.
- Recall spam **0,919**: bắt được 137 / 149 tin spam, để lọt 12.

Tất cả chỉ với vài dòng code, huấn luyện **dưới 1 giây**.

## 5.2. Thử với tin nhắn mới

```
moi = ["Congratulations! You have WON a free iPhone. Call now to claim your
prize",
       "Hey, are we still meeting for lunch tomorrow?",
       "URGENT: your account will be suspended, reply with your password"]
print(nb.predict(vec.transform(moi)))                    # nhớ: transform, KHÔNG
```

```
fit lại
print(nb.predict_proba(vec.transform(moi)).round(4))      # cột 0: ham, cột 1:
spam
```

**Kết quả mẫu:**

```
['spam' 'ham' 'spam']
[[0.     1.    ]
 [1.     0.    ]
 [0.0109 0.9891]]
```

### 5.3. Đáp án phần "Tìm lỗi"

Dòng lỗi: `CountVectorizer().fit_transform(X_test)` tạo **một bộ từ vựng mới** từ tập test (3.590 từ), khác với bộ từ vựng của train (7.668 từ). Cột thứ 5 của train có thể là "free", nhưng cột thứ 5 của test lại là "lunch". Mô hình nhận vào một bảng với số cột khác và **ý nghĩa cột khác hẳn**.

**Sửa:** `X_test_so = vec.transform(X_test)`, dùng chính bộ `vec` đã fit trên train.

```
a = CountVectorizer().fit_transform(X_tr)     # từ vựng của train
b = CountVectorizer().fit_transform(X_te)     # ❌ từ vựng riêng của test
print(a.shape, b.shape)
```

**Kết quả mẫu:**

```
(4457, 7668) (1115, 3590)
```

Đây cũng là **quy tắc fit-trên-train** ở bài 1.11: mọi thứ "học" từ dữ liệu (μ, σ của scaler, bộ từ vựng của vectorizer...) **chỉ học từ train**. Từ nào trong test chưa có trong từ vựng train sẽ bị bỏ qua, và điều đó là đúng: ngoài đời bạn cũng không biết trước các từ của tin nhắn tương lai.

### 5.4. Mô hình đã học được gì?

```
tu = np.array(vec.get_feature_names_out())
chenh = nb.feature_log_prob_[1] - nb.feature_log_prob_[0]   # log P(từ|spam) − log
P(từ|ham)
print("Đặc trưng của spam:", tu[np.argsort(chenh)[-12:]][::-1])
print("Đặc trưng của ham: ", tu[np.argsort(chenh)[:10]])
```

**Kết quả mẫu:**

```
Đặc trưng của spam: ['claim' 'prize' '150p' 'tone' '18' 'cs' '500' 'guaranteed'
'1000' 'uk'
 'landline' '150ppm']
Đặc trưng của ham:  ['gt' 'lt' 'he' 'da' 'lor' 'she' 'later' 'amp' 'll' 'ask']
```

- Spam: "claim", "prize", "guaranteed", các con số tiền "500", "1000", giá cước "150p"... Rất đúng chất tin rác.
- Ham: từ đời thường "he", "she", "later", "ask", tiếng lóng Singapore "lor", "da". "gt", "lt", "amp" là mã ký tự HTML (`>`, `<`, `&`) do lỗi xuất dữ liệu: một ví dụ nhỏ về dữ liệu "bẩn" (bài 1.6).

### 5.5. Ảnh hưởng của α

```python
for a in [0.01, 0.1, 1.0, 5.0]:
    acc = accuracy_score(y_te, MultinomialNB(alpha=a).fit(Xtr, y_tr).predict(Xte))
    print(a, round(acc, 4))
```

**Kết quả mẫu:**

```
0.01 0.9874
0.1 0.9883
1.0 0.9874
5.0 0.9713
```

α quá lớn (5) làm mô hình "san phẳng" xác suất các từ quá nhiều, giảm khả năng phân biệt. α là một tham số cần chọn (bài 2.18).

---

## 6. Các biến thể

| Biến thể | Dữ liệu phù hợp | Ví dụ |
|---|---|---|
| `MultinomialNB` | **Số đếm** (số lần từ xuất hiện) | Phân loại văn bản (bài này) |
| `BernoulliNB` | Có/không (0/1) | "Từ này có xuất hiện không" |
| `GaussianNB` | Số liên tục, giả định phân phối chuẩn (bài 1.7) | Đo đạc y tế, cảm biến |

---

## 7. Ưu, nhược điểm

| Ưu điểm | Nhược điểm |
|---|---|
| Cực nhanh cả khi huấn luyện lẫn dự đoán | Giả định độc lập thường sai → xác suất ra **quá tự tin** (thường gần 0 hoặc 1, như mục 5.2) |

| Ưu điểm | Nhược điểm |
|---|---|
| Tốt với ít dữ liệu và rất nhiều đặc trưng (văn bản) | Không học được tương tác giữa các từ ("not good" ≠ "good") |
| Dễ hiểu, dễ giải thích | Với dữ liệu dạng bảng, thường thua các mô hình khác |
| Mốc (baseline) tuyệt vời cho mọi bài toán văn bản | |

## 8. Bài tập

1. **Tính tay** với dữ liệu đồ chơi ở mục 4.4: tin nhắn **"meet tomorrow"** là spam hay ham? Xác suất bao nhiêu? Kiểm tra bằng code.
2. Thử `CountVectorizer(stop_words="english")` (bỏ các từ quá phổ biến như "the", "is"). Accuracy và recall spam thay đổi thế nào?
3. Thử `CountVectorizer(ngram_range=(1, 2))`: đếm cả **cặp từ liền nhau** ("free entry", "call now"). Số cột tăng lên bao nhiêu? Kết quả có tốt hơn không?
4. Tìm vài tin **spam bị bỏ lọt** (FN) trong tập test và đọc chúng. Vì sao mô hình không bắt được?

### Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. P(meet|spam) = (0+1)/18 ≈ 0,0556; P(tomorrow|spam) = 1/18 ≈ 0,0556. P(meet|ham) = (2+1)/18 ≈ 0,1667; P(tomorrow|ham) = 3/18 ≈ 0,1667.
   - Điểm spam = 0,5 × 0,0556 × 0,0556 ≈ 0,001543
   - Điểm ham = 0,5 × 0,1667 × 0,1667 ≈ 0,013889
   - P(ham) = 0,013889 / 0,015432 = **0,9** → **ham**. Kiểm tra: `m.predict_proba(cv.transform(["meet tomorrow"]))` cho `[[0.9 0.1]]`.
2. Code:

```
vec2 = CountVectorizer(stop_words="english")
Xtr2, Xte2 = vec2.fit_transform(X_tr), vec2.transform(X_te)
d2 = MultinomialNB().fit(Xtr2, y_tr).predict(Xte2)
print(classification_report(y_te, d2, digits=3))
```

So sánh với mục 5.1 và ghi lại. Với tin nhắn ngắn, bỏ stopword không phải lúc nào cũng giúp: đôi khi chính những từ nhỏ như "you", "your" lại là dấu hiệu của spam. 3. Code tương tự với `ngram_range=(1, 2)`, in `Xtr2.shape` để xem số cột (tăng lên nhiều lần). Cặp từ giúp mô hình nắm được cụm như "call now", "claim your". Ghi lại kết quả thật để so sánh. 4. Code:

```
bo_lot = X_te[(y_te == "spam") & (d == "ham")]
for tin in bo_lot.head(5):
    print("-", tin)
```

Thường là những tin spam **viết giống tin bình thường**, ít từ "nhạy cảm", hoặc rất ngắn. Đây là giới hạn của cách đếm từ: không hiểu được ý nghĩa câu. Các mô hình hiểu ngữ cảnh như BERT (bài NLP3.3) làm tốt hơn.

---

## 9. Flashcard

| Mặt trước | Mặt sau |
|---|---|
| Naive Bayes dựa trên gì? | Định lý Bayes + giả định các đặc trưng độc lập khi biết lớp. |
| Vì sao gọi là "ngây thơ"? | Giả định các từ độc lập, dù thực tế không phải vậy. |
| Bag of Words là gì? | Biểu diễn văn bản bằng số lần xuất hiện mỗi từ, bỏ qua thứ tự. |
| Laplace smoothing để làm gì? | Tránh xác suất 0 cho từ chưa gặp, vì nhân với 0 sẽ xóa sạch điểm. |
| `fit` của CountVectorizer học gì? | Bộ từ vựng. |
| Lỗi khi fit vectorizer riêng cho test? | Số cột và ý nghĩa cột khác train → sai hoặc báo lỗi. |
| Ma trận thưa là gì? | Ma trận gần như toàn 0; chỉ lưu các ô khác 0. |
| MultinomialNB vs GaussianNB? | Multinomial: số đếm (văn bản). Gaussian: số liên tục. |
| Nhược điểm chính về xác suất? | Thường quá tự tin (gần 0 hoặc 1). |

**Bài tiếp theo:** 2.9 – Decision Tree.